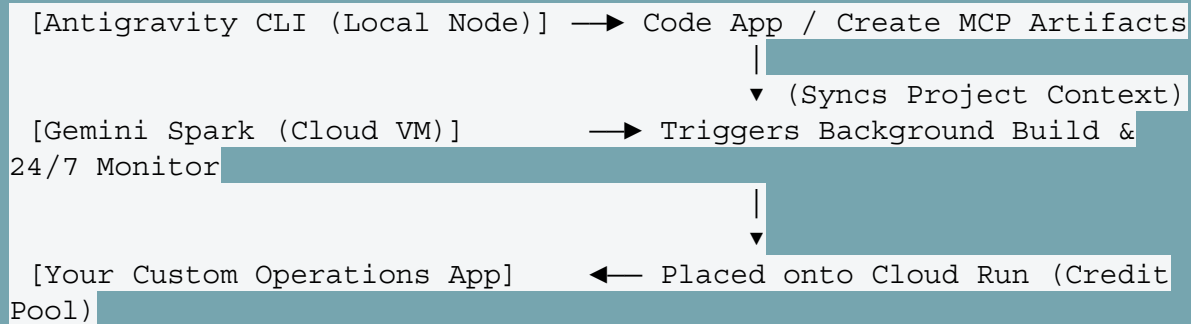


The combination of **Antigravity CLI** and **Gemini Spark** is exactly how you orchestrate your asynchronous "Captain's Chair" agent stack to build this app. [1, 2]

Because they share the exact same underlying core **Antigravity agent harness**, they can seamlessly hand off tasks to one another across your network. [3]

How Antigravity and Gemini Spark Work Together to Build Your App

The workflow uses Antigravity locally on your node cluster to write code, while Gemini Spark runs 24/7 in the cloud to manage long-running background tasks like deployment or monitoring your data stores. [1, 4, 5]



1. **Local Scaffolding with Antigravity CLI (agy):** You use the Go-based [Antigravity CLI](#) right inside your terminal to layout your MCP server files and the core structure of your Help Desk application. It uses **Asynchronous Subagents** to code features and build files in parallel without freezing your shell. [3, 5, 6]
2. **Handoff to Gemini Spark:** Because you want an agent that builds your tools for you, you can hand the local project context up to **Gemini Spark**. Spark lives on persistent Google Cloud virtual machines, meaning it keeps processing your code, building containers, or testing your database routing loops 24/7 even after you shut down your terminal or lock your phone. [1, 7, 8, 9]
3. **The Deployment App Hub:** Spark automates pushing your finalized container files straight to **Cloud Run**, creating a permanently active custom server app that securely draws from your \$1,000 credit pool. [1, 9]

Will Speech and Vision Break the Rules?

Direct Answer: Absolutely not. Adding Speech and Vision will not break any platform or credit constraints.

The Antigravity engine natively supports multi-modal image inputs, and the **Gemini Enterprise Agent Platform** features full support for multi-modal grounding. [10, 11]

If your **Operational Guide** app needs to process vision (like code diagrams, layout screenshots, or hardware blueprints) or speech (voice descriptions of user manuals), it falls entirely under Google Cloud's permitted enterprise billing tracks. [1, 2, 11, 12]

The Multi-Modal Rules Checklist:

-
- **Vision is 100% Covered:** When you link a Vertex AI Data Store to a Google Cloud Storage bucket containing image-heavy manuals, PDFs, or system diagrams, the token verification is processed through the standard Agent Builder track, safely using your credits. [11, 12]
- **Speech is 100% Covered:** Integrating voice features using the Vertex AI Voice/ADK (Agent Development Kit) pipeline allows your Help Desk agent to communicate hands-free, keeping billing confined to your cloud project's enterprise boundary. [13]
-

Step 1: Initialize Your Tool-Building Agent Workspace

To get your Antigravity CLI configured to start generating your app files, run this initial setup inside your terminal workspace to establish a clean environment: [14]

```
# Ensure you are on the latest Go-based Antigravity client
agy /config --trust-workspace true
```

```
# Open the multi-agent management matrix
/agents
```

Step 2: The Core Prompt for the Tool-Builder

Once your local workspace is initialized, paste this precise instruction block into your **Antigravity CLI** or **Gemini Spark** thread to kick off the "Captain's Chair" app-building workflow:

```
Initialize a new application build project named
"Neuromesh_Command_Hub".
```

```
You are an expert infrastructure architect task with building a
developer help desk and operational guide server app.
```

Generate the following core code artifacts:

1. An implementation plan tracking file formats for model weights, software manuals, and custom Model Context Protocol (MCP) server endpoints.
2. A containerized Python backend that maps local database snapshots and tool logs into an automated, multi-modal ingestion schema (supporting inline Base64 images for Vision and real-time text logs).
3. A deployment script targeting Google Cloud Run to allow this server tool to host the operational manuals on enterprise credits.

Utilize your background subagents to code the files in parallel and test the UI components inside the sandboxed web browser before compiling. Avoid all conversational filler—return only clean schemas and production code.

By firing this prompt, the **Antigravity harness** will spin up subagents to write the code, while **Gemini Spark** keeps the project running in the background until the full local-to-cloud server infrastructure is built, tested, and deployed to your Cloud Run terminal. [1, 9, 15]

Would you like to write the **Apache Spark config lines** to manage your data pipeline indexing, or should we draft the **MCP tool schemas** for the local Jetson to handle the app communication?

- [1] <https://www.youtube.com>
- [2] <https://www.youtube.com>
- [3] <https://developers.googleblog.com>
- [4] <https://blog.google>
- [5] <https://antigravity.google>
- [6] <https://dev.to>
- [7] <https://blog.google>
- [8] <https://sites.duke.edu>
- [9] <https://dev.to>
- [10] <https://ai.google.dev>
- [11] <https://www.youtube.com>
- [12] <https://cloud.google.com>
- [13] <https://ai.plainenglish.io>
- [14] <https://codelabs.developers.google.com>
- [15] <https://www.youtube.com>